

API Architecture

MVC, Routing, and REST Design

The GiftList API follows standard REST conventions with Laravel's MVC architecture. This makes the API predictable for consumers and the codebase navigable for developers.

The MVC Pattern

Models (app/Models/) — Define data structure and relationships:

- `Gift.php` — Represents a gift record
- `Recipient.php` — Represents a recipient record
- `User.php` — Represents an authenticated user

Controllers (app/Http/Controllers/) — Handle requests and return responses:

- `GiftController.php` — CRUD operations for gifts
- `RecipientController.php` — CRUD operations for recipients
- `AuthController.php` — Login, logout, registration

Routes (routes/api.php) — Map URLs to controller methods

No business logic in routes. No database queries in controllers (ideally). Each layer has one job.

RESTful Endpoints

The API follows REST conventions:

Method	Endpoint	Action
GET	<code>/api/gifts</code>	List all gifts
POST	<code>/api/gifts</code>	Create a gift
GET	<code>/api/gifts/{id}</code>	Get one gift
PUT	<code>/api/gifts/{id}</code>	Update a gift
DELETE	<code>/api/gifts/{id}</code>	Delete a gift

Same pattern for recipients. Consumers know what to expect without reading documentation.

Middleware

Middleware handles cross-cutting concerns—things that apply to many routes:

```
Route::middleware('auth:sanctum')->group(function () {
    Route::apiResource('gifts', GiftController::class);
    Route::apiResource('recipients', RecipientController::class);
});
```

Authentication is declared once, applied everywhere. No repetitive checks in every controller method.

Response Consistency

Laravel's API Resources transform models into consistent JSON:

```
{
  "data": {
    "id": 1,
    "name": "Book",
    "recipient": {
      "id": 5,
      "name": "Alice"
    }
  }
}
```

Same structure everywhere. Clients can rely on it.

Why This Matters

This architecture isn't clever—it's conventional. That's the point:

- New developers understand it immediately
- Standard tools (Postman, API clients) work out of the box
- Changes are isolated to the appropriate layer
- Testing is straightforward

The GiftList API isn't impressive because it's complex. It's useful because it's simple and follows established patterns.